

NAKAST

Código Fuente

UNIVERSIDAD DE VALPARAÍSO

Autores:

JAVIER ALEJANDRO TOGNARELLI SANTIAGO

DANIEL FERNANDO ESCOBAR ARAYA

FERNANDO ANDRÉS AMAYA INZUNZA

SEPTIEMBRE 2026

Contenido

Este documento reproduce íntegramente el código fuente de NAKAST: el flujo de trabajo en Nextflow, su configuración y los scripts en Python que ejecuta. Se omiten la documentación, el catálogo de especies compatibles (`assets/pubmlst_species.tsv`, que contiene datos y no código) y los scripts que generan la documentación en PDF.

Repositorio: <https://github.com/javiertognarelli/nakast>. Estado del código listado: commit `bfedcd3`.

La huella SHA-256 de cada archivo permite verificar que una copia del código coincide exactamente con la aquí listada.

#	Archivo	Descripción	Líneas	SHA-256
1	<code>nakast.nf</code>	Definición del flujo de trabajo y de sus procesos (Nextflow)	503	<code>ef428fefbeeb786f335fab a8706aa37a4b86c3ea217c af62a1ce91c022e80827</code>
2	<code>nextflow.config</code>	Configuración de parámetros, recursos, ambientes y perfiles	113	<code>253b777170301843dd994b1 4bf947b54250a078e6aa568 c414e9830ded59b2d1</code>
3	<code>bin/validate_samplesheet.py</code>	Validación de la tabla de muestras	113	<code>4727a1169fd7ddd0f39da17 166471dd8fed0b5e537d241 48d499e7c0b515dc44</code>
4	<code>bin/pubmlst_api.py</code>	Cliente de la API REST de PubMLST	146	<code>c41233a2c6b53927a53306 d1ff19d7f732367b0a13c51 65b44ce96b8552652fb</code>
5	<code>bin/download_pubmlst.py</code>	Descarga del esquema MLST de cada especie	139	<code>fd5f392c258c372de34bff9 35749e414c80814b3542c74 ce79cef91eaace9713</code>
6	<code>bin/parse_kma.py</code>	Llamado de alelos y asignación de ST y CC	287	<code>3222d0c7021b2a933dcf1c de2a741a7f3d72588153c21 55a73a1af86890747a4</code>
7	<code>bin/list_pubmlst_species.py</code>	Catálogo de especies compatibles	147	<code>737db11a6337daf224d2d83 41e1884716e63ee7a1807f9 aa411295a51c99f2e7</code>

Total: 7 archivos, 1448 líneas.

1. nakast.nf

Definición del flujo de trabajo y de sus procesos (Nextflow) · 503 líneas · SHA-256
ef428fefbeeb786f335faba8706aa37a4b86c3ea217caf62a1ce91c022e80827

```
1  #!/usr/bin/env nextflow
2
3  // NAKAST - MLST typing from Oxford Nanopore amplicon sequencing.
4  //
5  // Designed and validated with Streptococcus agalactiae. The species is a
6  // per-sample parameter, so the pipeline works with any MLST scheme published
7  // by PubMLST; see docs/supported_species.md for the catalogue and
8  // docs/USER_MANUAL.md for design rationale, assumptions and limitations.
9  //
10 // Authors:
11 //   Javier Tognarelli Santiago
12 //   Genomica UV, Escuela de Medicina, Universidad de Valparaiso, Chile
13 //   Daniel Escobar Araya
14 //   Unidad de Investigacion e Innovacion, Instituto de Salud Publica, Chile
15 //   Fernando Amaya Inzunza
16 //   Unidad de Investigacion e Innovacion, Instituto de Salud Publica, Chile
17 //
18 // Created: December 2025
19
20 nextflow.enable.dsl=2
21
22 // Run-level inputs and outputs
23 params.samplesheet = 'samplesheet.tsv'
24 params.outdir = 'results'
25
26 // Analysis parameters
27 params.nano_base_quality = 20
28 params.mlst_report = "mlst_profiles"
29 params.min_depth = 30
30 params.allele_coverage = 90
31 params.min_identity = 90
32 params.min_length = 300
33 params.keep_percent = '80'
34
35 // Read quality is enforced by chopper alone. filtlong's --min_mean_q and
36 // --min_window_q are deliberately left unset, for two reasons:
37 //   1. They use a 0-100 accuracy scale rather than Phred, so passing a Phred
38 //   value filters nothing at all (Q20 == 99, Q25 == 99.7, Q30 == 99.9).
39 //   2. Setting them to an equivalent threshold removes weakly amplified loci
40 //   outright: low-abundance loci also carry the lowest-quality reads, and
41 //   benchmarking showed such loci falling from >30x depth to zero.
42
43 params.generate_consensus = true
44 params.qc_outdir = "${params.outdir}/qc"
45 params.consensus_outdir = "${params.outdir}/consensus_sequences"
46 params.report_outdir = "${params.outdir}/reports"
47
48 params.run_name = 'NAKAST_Run'
49 params.run_description = 'NAKAST: MLST typing from Oxford Nanopore reads, for any PubMLST
50 scheme'
51
52 // Catalogue of supported schemes, used to validate the 'species' column
53 params.species_catalogue = "$projectDir/assets/pubmlst_species.tsv"
54 params.kma_k = 31
55
56 // PubMLST access. By default a data access key is required, read from the
57 // Nextflow secrets store as PUBMLST_API_KEY. Anonymous access returns only data
58 // deposited up to 31 December 2024, so it must be requested explicitly.
59 params.pubmlst_anonymous = false
60
61 params.qc_only = false
62 params.list_species = false
63 params.help = false
```

```

63
64 def print_help() {
65     log.info """
66         Usage:
67         nextflow run nakast.nf [options]
68
69         Description:
70         NAKAST determines MLST profiles from Oxford Nanopore reads. The species
71         is declared per sample in the 'species' column of the samplesheet, so a
72         single batch may mix species.
73
74         Main options:
75         --samplesheet      Input sample table (default: samplesheet.tsv)
76         --outdir           Base output directory (default: results)
77
78         MLST options:
79         --mlst_report      Base name of the MLST report (default: mlst_profiles)
80         --min_depth        Minimum depth to call an allele (default: 30)
81         --min_identity     Minimum identity (%) (default: 90)
82         --allele_coverage  Minimum allele coverage (%) (default: 90)
83         --generate_consensus Write per-locus consensus sequences (default: true)
84
85         Read filtering options:
86         --nano_base_quality Minimum Phred base quality (default: 20)
87         --min_length        Minimum read length (default: 300)
88         --keep_percent      Keep this percentage of the best reads (default: 80)
89
90         Output directories:
91         --qc_outdir         Read QC results
92         (default: results/qc)
93
94         --consensus_outdir  Consensus sequences
95         (default: results/consensus_sequences)
96
97         --report_outdir     Alignment results and profile tables
98         (default: results/reports)
99
100
101         Run modes:
102         --qc_only           Run quality control only (default: false)
103         --list_species      List the supported PubMLST species and exit
104
105         PubMLST access:
106         A PubMLST data access key is required. Store it once with:
107             nextflow secrets set PUBMLST_API_KEY <key>
108         --pubmlst_anonymous Run without a key; only data deposited up to
109             31 December 2024 is used (default: false)
110
111         Other:
112         --kma_k             K-mer size for the KMA index (default: 31)
113         --help              Print this message and exit
114
115         Samplesheet (TSV with header):
116         sample_id fastq_dir species
117         M1         data/M1  sagalactiae
118         M2         data/M2  salmonella
119
120         The 'species' column takes the names listed in
121         docs/supported_species.md (135 schemes).
122
123         Example:
124         nextflow run nakast.nf --samplesheet samples.tsv --outdir results --min_depth 40
125
126         """.stripIndent()
127     }
128     workflow MLST_TYPING {
129         main:
130         // 1. Resolve inputs
131         ch_input = file(params.samplesheet, checkIfExists: true, followLinks: true)
132         ch_catalogue = file(params.species_catalogue, checkIfExists: true)
133     }

```

```

134 // 2. Pre-flight validation. Every downstream channel derives from
135 // its output, so an invalid samplesheet halts the run before any
136 // expensive work starts.
137 VALIDATE_SAMPLESHEET(ch_input, "${launchDir}", ch_catalogue)
138
139 // 3. Build the sample channel. Database downloads are launched
140 // further down, once the species present in the run are known.
141 VALIDATE_SAMPLESHEET.out.ok
142   .map { file(params.samplesheet, checkIfExists: true, followLinks: true) }
143   .splitCsv(header:true, sep:'\t')
144   .map { row ->
145     def sample_info = [id: row.sample_id, species: row.species.trim()]
146     return [ sample_info, file(row.fastq_dir, checkIfExists: true, followLinks: tr
ue)]
147   }
148   .set { ch_reads }
149
150 ch_reads
151   | concat_ONT_fastq
152   | FILTER_ONT
153
154 nanoplot_out = NANOPLOT(FILTER_ONT.out.filtered_reads)
155 qc_dirs_ch = nanoplot_out.map { sample, qc_dir -> qc_dir }.collect()
156 MULTIQC(qc_dirs_ch) // aggregate every per-sample QC folder in one report
157
158 // One database per distinct species; 'unique' avoids re-downloading
159 // the same scheme for every sample that shares it.
160 ch_species = ch_reads.map { sample, _reads -> sample.species }.unique()
161 def db = params.pubmlst_anonymous
162       ? DOWNLOAD_PREP_DB_ANONYMOUS(ch_species)
163       : DOWNLOAD_PREP_DB(ch_species)
164
165 // Join each sample to the database built for its own species.
166 ch_kma = FILTER_ONT.out.filtered_reads
167   .map { sample, reads -> [ sample.species, sample, reads ] }
168   .combine(db.mlst_db, by: 0)
169 KMA_RUN(ch_kma)
170
171 // One report per species: loci differ between schemes and cannot
172 // share a single table of columns.
173 ch_report = KMA_RUN.out.kma_result
174   .groupTuple()
175   .join(db.mlst_profiles)
176 GENERATE_REPORT(ch_report)
177 }
178 workflow list_species {
179   main:
180     LIST_SPECIES()
181 }
182 workflow qc_only {
183   main:
184     ch_input = file(params.samplesheet, checkIfExists: true, followLinks: true)
185     ch_catalogue = file(params.species_catalogue, checkIfExists: true)
186
187     // Same pre-flight validation as MLST_TYPING: the read channel derives
188     // from its output, so nothing runs on an invalid samplesheet.
189     VALIDATE_SAMPLESHEET(ch_input, "${launchDir}", ch_catalogue)
190
191     VALIDATE_SAMPLESHEET.out.ok
192       .map { file(params.samplesheet, checkIfExists: true, followLinks: true) }
193       .splitCsv(header:true, sep:'\t')
194       .map { row ->
195         def sample_info = [id: row.sample_id, species: row.species.trim()]
196         return [ sample_info, file(row.fastq_dir, checkIfExists: true, followLinks: tr
ue)]
197       }
198       .set { ch_reads }
199
200 ch_reads
201   | concat_ONT_fastq
202   | FILTER_ONT

```

```

203     nanoplots_out = NANOPLOT(FILTER_ONT.out.filtered_reads)
204     qc_dirs_ch = nanoplots_out.map { sample, qc_dir -> qc_dir }.collect()
205     MULTIQC(qc_dirs_ch) // aggregate every per-sample QC folder in one report
206 }
207
208 workflow {
209     // Handled inside the workflow rather than at script level: Nextflow 26's
210     // strict syntax requires statements to live in a process, workflow or
211     // function.
212     if (params.help) {
213         print_help()
214         return
215     }
216
217     // Fail before any work starts when typing would need a key that is not
218     // there. QC-only and species listing do not download scheme data.
219     def needs_download = !params.list_species && !params.qc_only
220     if (needs_download && !params.pubmlst_anonymous && !params.pubmlst_key_found) {
221         error '''
222             No PubMLST API key found.
223
224             NAKAST needs a PubMLST data access key to obtain the complete MLST
225             scheme; without one, PubMLST only serves data deposited up to
226             31 December 2024. Create a key in your PubMLST account and store it:
227
228             nextflow secrets set PUBMLST_API_KEY <key>
229
230             To run anyway with data up to 2024 only, add --pubmlst_anonymous.
231             '''.stripIndent()
232     }
233
234     if (params.list_species) {
235         list_species()
236     }
237     else if (params.qc_only) {
238         qc_only()
239     }
240     else {
241         MLST_TYPING()
242     }
243 }
244
245 process VALIDATE_SAMPLESHEET {
246     tag "validacion"
247     conda 'conda-forge::python=3.13.11 conda-forge::pandas=2.3.3'
248     label 'process_low'
249     // Overrides the global 'ignore' strategy: an invalid samplesheet must
250     // stop the run rather than be skipped silently.
251     errorStrategy 'terminate'
252
253     input:
254         path samplesheet
255         // launchDir is passed as a String instead of being interpolated into
256         // the script body. Interpolating the Path object puts it in the task's
257         // hashed context, where its hash is unstable across runs and breaks
258         // -resume caching.
259         val basedir
260         // Scheme catalogue, so an unknown species fails here rather than
261         // after a partial download.
262         path catalogue
263
264     output:
265         path "validacion.ok", emit: ok
266
267     script:
268         // Redirection rather than a pipe: Nextflow runs with `bash -ue` and
269         // without pipefail, so a pipe would mask the script's exit code.
270         """
271         validate_samplesheet.py ${samplesheet} --basedir ${basedir} \\\
272         --catalogue ${catalogue} > validacion.ok \\\
273         || { cat validacion.ok >&2; exit 1; }
274         cat validacion.ok
275         """

```

```

274 }
275 // Script shared by both download processes. Only the access mode differs.
276 def download_script(species, anonymous) {
277     """
278     download_pubmlst.py --species ${species} --outdir . ${anonymous ? '--anonymous' : ''}
279
280     kma index -i ${species}_alleles.fasta -k ${params.kma_k} -o ${species}_db
281     """
282 }
283
284 process DOWNLOAD_PREP_DB {
285     tag "$species"
286     conda 'conda-forge::python=3.13.11 bioconda::kma=1.6.8'
287     label 'process_low'
288     publishDir "${params.report_outdir}", mode: 'copy', pattern: "*_profiles.tsv"
289
290     // Exposed to the task as an environment variable. Nextflow keeps secrets
291     // out of the work directory, the command line and the logs.
292     secret 'PUBMLST_API_KEY'
293
294     // Overrides the global 'ignore' strategy: without a database there is no
295     // typing, so a skipped failure would yield a run with no results.
296     // Exit code 3 means the key is missing or was rejected, which retrying
297     // cannot fix. Any other failure is treated as a transient network fault:
298     // retried, then 'finish' lets in-flight QC complete before stopping, so a
299     // later -resume reuses it and repeats only the download.
300     errorStrategy { task.exitStatus == 3 ? 'terminate' : (task.attempt <= 3 ? 'retry' : 'finis
301 h') }
302     maxRetries 3
303
304     input:
305         val species
306     output:
307         tuple val(species), path("${species}_profiles.tsv"), path("${species}_loci.txt"), emit
308 : mlst_profiles
309         tuple val(species), path("${species}_db.*"), emit: mlst_db
310     script:
311         download_script(species, false)
312 }
313
314 // Same as DOWNLOAD_PREP_DB without the secret, for --pubmlst_anonymous. A
315 // separate process is needed because declaring a secret that does not exist
316 // aborts the task, and the directive cannot be made conditional.
317 process DOWNLOAD_PREP_DB_ANONYMOUS {
318     tag "$species"
319     conda 'conda-forge::python=3.13.11 bioconda::kma=1.6.8'
320     label 'process_low'
321     publishDir "${params.report_outdir}", mode: 'copy', pattern: "*_profiles.tsv"
322
323     errorStrategy { task.attempt <= 3 ? 'retry' : 'finish' }
324     maxRetries 3
325
326     input:
327         val species
328     output:
329         tuple val(species), path("${species}_profiles.tsv"), path("${species}_loci.txt"), emit
330 : mlst_profiles
331         tuple val(species), path("${species}_db.*"), emit: mlst_db
332     script:
333         download_script(species, true)
334 }
335
336 process concat_ONT_fastq {
337     tag "$sample.id"
338     conda 'conda-forge::python=3.13.11 conda-forge::coreutils=9.5 conda-forge::gzip=1.12'
339     // A failed sample is marked as such and the rest of the batch continues
340     errorStrategy 'ignore'
341     label 'process_medium'
342
343     input:
344         tuple val(sample), path(input_dir)
345     output:

```

```

342     tuple val(sample), path("${sample.id}.unfiltered.fastq.gz"), emit: unfiltered_reads
343   script:
344     """
345     if ls ${input_dir}/*.fastq.gz > /dev/null 2>&1; then
346       echo "Concatenating compressed files..."
347       cat ${input_dir}/*.fastq.gz > ${sample.id}.unfiltered.fastq.gz
348
349     # Fall back to uncompressed .fastq
350     elif ls ${input_dir}/*.fastq > /dev/null 2>&1; then
351       cat ${input_dir}/*.fastq | gzip > ${sample.id}.unfiltered.fastq.gz
352
353     # Neither present: fail this sample only
354     else
355       # Surfaced in .nextflow.log
356       echo "ERROR: sample ${sample.id} is empty or malformed" >&2
357       # errorStrategy 'ignore' confines the failure to this sample
358       exit 1
359     fi
360     """
361   }
362   process FILTER_ONT {
363     tag "$sample.id"
364     conda 'conda-forge::python=3.13.11 bioconda::filtlong=0.3.1 bioconda::chopper=0.12.0
365     conda-forge::gzip=1.12'
366     // Intermediate output, not published. Uncomment to keep the filtered reads:
367     // publishDir "${params.outdir}/ont_filtered", mode: 'copy'
368     label 'process_medium'
369
370     input:
371     tuple val(sample), path(reads)
372
373     output:
374     tuple val(sample), path("${sample.id}.fastq.gz"), emit: filtered_reads
375
376     script:
377     """
378     # filtlong: length filter, then keep the best params.keep_percent of reads
379     #             (its ranking weights mean and window quality).
380     # chopper: Phred quality filter; splits reads at low-quality stretches.
381
382     filtlong --keep_percent ${params.keep_percent} \
383       --min_length ${params.min_length} \
384       --mean_q_weight 5 \
385       --window_q_weight 10 --window_size 50 ${reads} |
386     chopper -q ${params.nano_base_quality} --minlength 50 --trim-approach split-by-low-quality
387     --cutoff ${params.nano_base_quality} |
388     gzip > ${sample.id}.fastq.gz
389     """
390   }
391   process NANOPLLOT {
392     tag "$sample.id"
393     conda 'conda-forge::python=3.13.11 bioconda::nanoplot=1.46.2'
394     publishDir "${params.qc_outdir}", mode: 'copy'
395     label 'process_medium'
396
397     input:
398     tuple val(sample), path(reads)
399
400     output:
401     tuple val(sample), path("qc_${sample.id}")
402
403     script:
404     """
405     NanoPlot \
406       --fastq ${reads} \
407       --outdir qc_${sample.id} \
408       --prefix ${sample.id}_ \
409       --threads ${task.cpus} \
410       --tsv_stats
411     """
412   }

```

```

411 process MULTIQC {
412     tag "multiqc"
413     conda 'conda-forge::python=3.13.11 bioconda::multiqc=1.33'
414     publishDir "${params.qc_outdir}", mode: 'copy'
415     label 'process_medium'
416
417     input:
418         // All qc_* folders at once, so MultiQC runs a single time
419         path(qc_dirs)
420     output:
421         path "multiqc_report.html"
422         path "multiqc_data"
423         path "multiqc_plots"
424     script:
425         """
426         multiqc --force --interactive -p -v ${qc_dirs} -o .
427         """
428 }
429 process KMA_RUN {
430     tag "$sample.id ($species)"
431     conda 'conda-forge::python=3.13.11 bioconda::kma=1.6.8'
432     publishDir "${params.report_outdir}", mode: 'copy', pattern: "*.res"
433     label 'process_high'
434     // Overrides the global 'ignore' strategy: a failure here leaves the run
435     // without MLST results, and 'ignore' would exit 0 with no report and no
436     // warning.
437     errorStrategy 'terminate'
438
439     // 'enabled' rather than an if() block: Nextflow 26's strict syntax does
440     // not allow conditionals inside a process body.
441     publishDir "${params.consensus_outdir}", mode: 'copy', pattern: "*.fsa", enabled: params.g
442     erate_consensus
443
444     input:
445         tuple val(species), val(sample), path(reads), path(mlst_db_files)
446     output:
447         tuple val(species), path("${sample.id}.res"), emit: kma_result
448         path "${sample.id}.fsa", emit: consensus_sequences
449     script:
450         // allele_coverage is intentionally NOT passed to -mrc. That flag is
451         // "minimum query coverage", the aligned fraction of the READ, not of
452         // the allele. With reads roughly twice the amplicon length it discards
453         // loci that match the reference perfectly. Allele coverage is enforced
454         // downstream in parse_kma.py via Template_Coverage.
455         coverage = params.allele_coverage / 100
456         """
457         kma -i ${reads} -t_db ${species}_db -nf -t ${task.cpus} \\\
458             -ID ${params.min_identity} -lt1 \\\
459             -mrs ${coverage} -bc ${coverage} -eq ${params.nano_base_quality} -bcNano -lc \\\
460             -mq 30 -md ${params.min_depth} -bcd ${params.min_depth} -mi ${params.min_depth} \\\
461             -ef \\\
462             -reward 2 -penalty 6 -gapopen 15 -gapextend 3 -Npenalty 5 -o ${sample.id}
463         """
464 }
465 process GENERATE_REPORT {
466     tag "$species"
467     conda 'conda-forge::python=3.13.11 conda-forge::pandas=2.3.3 conda-forge::numpy=2.4.0
468     conda-forge::openpyxl=3.1.5'
469     publishDir "${params.outdir}", mode: 'copy', pattern: "${params.mlst_report}_"
470     label 'process_medium'
471     // Overrides the global 'ignore' strategy: a failure here leaves the run
472     // without MLST results, and 'ignore' would exit 0 with no report and no
473     // warning.
474     errorStrategy 'terminate'
475
476     input:
477         tuple val(species), path(kma_results), path(mlst_profiles), path(mlst_loci)
478     output:
479         path "${params.mlst_report}_${species}.txt"
480         path "${params.mlst_report}_${species}.xlsx"
481     script:

```

```

480     """
481     parse_kma.py -i . -p ${mlst_profiles} --loci ${mlst_loci} \\\
482         -o ${params.mlst_report}_${species} \\\
483         --min_depth ${params.min_depth} \\\
484         --min_identity ${params.min_identity} \\\
485         --min_coverage ${params.allele_coverage}
486     """
487 }
488 process LIST_SPECIES {
489     tag "pubmlst_catalogue"
490     conda 'conda-forge::python=3.13.11'
491     label 'process_low'
492     errorStrategy 'terminate'
493
494     output:
495         path "supported_species.txt"
496
497     script:
498         """
499         list_pubmlst_species.py --tabla > supported_species.txt \\\
500             || { cat supported_species.txt >&2; exit 1; }
501         cat supported_species.txt
502         """
503 }

```

2. nextflow.config

Configuración de parámetros, recursos, ambientes y perfiles · 113 líneas · SHA-256
253b777170301843dd994b14bf947b54250a078e6aa568c414e9830ded59b2d1

```
1  /*
2  * -----
3  * Nextflow config file for NAKAST
4  * -----
5  */
6
7  // 1. DEFAULT PARAMETERS
8  // Override any of these on the command line with --paramName value.
9  params {
10     // Input / output
11     samplesheet = "samplesheet.tsv"
12     outdir      = "results"
13
14     // Analysis parameters (Oxford Nanopore)
15     nano_base_quality = 20 // Phred 20, ~99% base accuracy
16
17     // General options
18     publish_dir_mode = 'copy' // 'copy', 'symlink', 'link'
19
20     // Whether a PubMLST API key is in the Nextflow secrets store. Only the
21     // boolean is kept here, never the key; nakast.nf uses it to fail early.
22     pubmlst_key_found = secrets.PUBMLST_API_KEY ? true : false
23
24     // Timestamp for the execution reports; see section 5
25     trace_timestamp = new java.util.Date().format('yyyy-MM-dd_HH-mm-ss')
26 }
27
28 // 2. PROCESS CONFIGURATION
29 process {
30
31     // --- Default error strategy ---
32     // A failing sample is skipped and the batch continues. Processes whose
33     // failure would invalidate the whole run override this individually.
34     errorStrategy = { task.exitStatus in [137, 140] ? 'retry' : 'ignore' }
35     maxRetries    = 2
36
37     // --- Default resources, used when a process declares no label ---
38     cpus    = 4
39     memory  = 6.GB
40     time    = 4.h
41
42     // --- Resources by label ---
43     // Processes select a tier with, for example, label 'process_low'.
44     withLabel: process_low {
45         cpus    = 2
46         memory  = 4.GB
47         time    = 2.h
48     }
49     withLabel: process_medium {
50         cpus    = 6
51         memory  = 10.GB
52         time    = 8.h
53     }
54     withLabel: process_high {
55         cpus    = 12
56         memory  = 16.GB
57         time    = 24.h
58     }
59
60     // --- Software provisioning ---
61     // Declared per process with the `conda` directive in nakast.nf; no
62     // container images are assigned here.
63 }
```

```

64
65 // 3. CONDA ENVIRONMENTS
66 // Enabled unconditionally. Each process declares its own pinned environment in
67 // nakast.nf; leaving this to a profile means that forgetting the flag silently
68 // runs every process against the host PATH, failing with exit 127.
69 // Note that keys inside the `conda {}` scope take no 'conda.' prefix: written
70 // as `conda.enabled` they resolve to 'conda.conda.enabled' and are ignored.
71 conda {
72     enabled      = true
73     useMamba     = true
74     cacheDir     = env('HOME') + "/.nextflow/conda"
75     createTimeout = '2h'
76 }
77
78 // 4. PROFILES
79 profiles {
80
81     // Default profile: local executor
82     standard {
83         process.executor = 'local'
84     }
85
86     // Kept for backwards compatibility. Conda is on by default, so this is
87     // redundant, but `-profile conda` must not error for existing scripts.
88     conda {
89         conda.enabled = true
90     }
91
92     // Escape hatch: use the tools on the host PATH instead of Conda.
93     host {
94         conda.enabled = false
95     }
96 }
97
98 // 5. EXECUTION REPORTS
99 // The timestamp lives in params rather than a `def`: Nextflow 26's strict
100 // config syntax rejects variable declarations ("Variable declarations cannot
101 // be mixed with config statements"). This form works on both 25.10 and 26.x.
102 timeline {
103     enabled = true
104     file    = "${params.outdir}/pipeline_info/execution_timeline_${
105         params.trace_timestamp}.html"
106 }
107 report {
108     enabled = true
109     file    = "${params.outdir}/pipeline_info/execution_report_${params.trace_timestamp}.html"
110 }
111 trace {
112     enabled = true
113     file    = "${params.outdir}/pipeline_info/execution_trace_${params.trace_timestamp}.txt"
114 }

```

3. bin/validate_samplesheet.py

Validación de la tabla de muestras · 113 líneas · SHA-256

4727a1169fd7ddd0f39da17166471dd8fed0b5e537d24148d499e7c0b515dc44

```
1  #!/usr/bin/env python3
2  import sys
3  import argparse
4  import pandas as pd
5  import os
6
7  def _resolve(p, basedir):
8      """Resolve relative samplesheet paths against basedir.
9
10     The script runs inside Nextflow's work directory, so relative paths must
11     be interpreted relative to the launch directory instead.
12     """
13     p = str(p).strip()
14     return p if os.path.isabs(p) else os.path.join(basedir, p)
15
16  def load_valid_species(catalogue):
17      """Read the 'species' column of assets/pubmlst_species.tsv."""
18      if catalogue is None:
19          return None
20      try:
21          df = pd.read_csv(catalogue, sep='\t', dtype=str)
22          return set(df['species'].str.strip())
23      except Exception as e:
24          print(f"[WARN] Could not read the species catalogue ({e}). "
25                f"Skipping validation of the 'species' column.", file=sys.stderr)
26          return None
27
28
29  def validate_samplesheet(file_path, basedir, valid_species=None):
30      errors = []
31
32      try:
33          df = pd.read_csv(file_path, sep='\t')
34      except Exception as e:
35          sys.exit(f"CRITICAL: could not read the samplesheet. {e}")
36
37      # 1. Required columns. Extra columns are ignored, so samplesheets written
38      # for earlier versions of the pipeline keep working.
39      required_cols = ['sample_id', 'fastq_dir', 'species']
40      missing_cols = set(required_cols) - set(df.columns)
41      if missing_cols:
42          sys.exit(f"ERROR: missing columns in the header: {missing_cols}")
43
44      # Normalise identifiers
45      df['sample_id'] = df['sample_id'].astype(str).str.strip()
46      df['species'] = df['species'].astype(str).str.strip()
47
48      # 2. sample_id must be present
49      if df['sample_id'].eq('nan').any() or df['sample_id'].eq('').any():
50          errors.append("Row(s) with an empty 'sample_id'.")
51
52      # sample_id names the output files, so it must be unique
53      if df['sample_id'].duplicated().any():
54          dups = df.loc[df['sample_id'].duplicated(), 'sample_id'].unique()
55          errors.append(f"Duplicate sample_id values: {dups}")
56
57      # 3. Per-row checks, iterated so each error names its line
58      for index, row in df.iterrows():
59          line_num = index + 2 # +2 porque index inicia en 0 y hay header
60          pid = row['sample_id']
61
62          # NAKAST processes Oxford Nanopore reads only: each sample points at a
63          # directory of FASTQ files, typically a barcode folder.
```

```

64     f_dir = row['fastq_dir']
65     if pd.isna(f_dir) or str(f_dir).strip() == '':
66         errors.append(f"[Row {line_num} - {pid}] Missing 'fastq_dir'.")
67     elif not os.path.exists(_resolve(f_dir, basedir)):
68         errors.append(f"[Row {line_num} - {pid}] fastq_dir does not exist: {f_dir}")
69
70     # Check the species against the catalogue so an unknown value fails in
71     # seconds rather than after a partial database download.
72     sp = row['species']
73     if pd.isna(sp) or str(sp).strip() in ('', 'nan'):
74         errors.append(f"[Row {line_num} - {pid}] Missing 'species'. "
75                       f"Run --list_species to see the supported values.")
76     elif valid_species is not None and str(sp).strip() not in valid_species:
77         suggestions = [e for e in sorted(valid_species) if e.startswith(str(sp).strip())[:
4]]][:5]
78         extra = f" Did you mean: {'', '.join(suggestions)}?" if suggestions else ""
79         errors.append(f"[Row {line_num} - {pid}] Species '{sp}' is not in the PubMLST "
80                       f"catalogue.{extra} Run --list_species for the full list.")
81
82     # 4. Report
83     if errors:
84         print("-----")
85         print("SAMPLESHEET ERRORS DETECTED:")
86         for e in errors:
87             print(f" - {e}")
88         print("-----")
89         sys.exit(1) # Report con error para detener Nextflow
90     else:
91         print(f"Samplesheet OK: {len(df)} samples found.")
92         sys.exit(0)
93
94 if __name__ == "__main__":
95     parser = argparse.ArgumentParser(
96         description="Validate the NAKAST samplesheet."
97     )
98     parser.add_argument("samplesheet", help="Samplesheet file (TSV)")
99     parser.add_argument(
100         "--basedir",
101         default=os.getcwd(),
102         help="Directory that relative samplesheet paths resolve against "
103              "(default: current directory)",
104     )
105     parser.add_argument(
106         "--catalogue",
107         default=None,
108         help="TSV of supported PubMLST species (assets/pubmlst_species.tsv). "
109              "If omitted, the 'species' column is not validated.",
110     )
111     args = parser.parse_args()
112     validate_samplesheet(args.samplesheet, args.basedir,
113                          load_valid_species(args.catalogue))

```

4. bin/pubmlst_api.py

Cliente de la API REST de PubMLST · 146 líneas · SHA-256

c41233a2c6b53927a53306d1ff19d7f732367b0a13c5165b44ce96b8552652fb

```
1  #!/usr/bin/env python3
2  """Minimal client for the PubMLST REST API.
3
4  Depends on the standard library only, so processes that import it do not need
5  extra packages in their Conda environment.
6
7  Two assumptions that hold for common schemes but not in general, and which this
8  module resolves dynamically instead:
9
10     * The MLST scheme is not always `schemes/1`. Of the 135 PubMLST schemes that
11       define ST profiles, seven use another id (Salmonella uses 2).
12       `detect_mlst_scheme()` finds it by description.
13     * Schemes do not always have seven loci; they range from 2 to 10. The locus
14       list is always read from the scheme definition.
15 """
16
17 import json
18 import os
19 import sys
20 import time
21 import urllib.error
22 import urllib.request
23
24 BASE_URL = "https://rest.pubmlst.org"
25
26 # PubMLST data access key. Without it the API serves only data deposited up to
27 # 31 December 2024; newer alleles and STs require an account. The key is read
28 # from the environment, where Nextflow places it from its secrets store, so it
29 # never appears in the code, the command line or the task work directory.
30 API_KEY_ENV = "PUBMLST_API_KEY"
31
32
33 class AuthenticationError(RuntimeError):
34     """The API key was rejected. Retrying cannot fix this."""
35
36
37 def authenticated():
38     """True when a data access key is available."""
39     return bool(os.environ.get(API_KEY_ENV, "").strip())
40
41
42 def _headers():
43     key = os.environ.get(API_KEY_ENV, "").strip()
44     return {"X-API-Key": key} if key else {}
45
46
47 # Retry policy for brief outages and transient 5xx responses from PubMLST.
48 ATTEMPTS = 5
49 BACKOFF_BASE = 5
50 TIMEOUT = 120
51
52
53 def _get(url, binary=False):
54     """GET with retries and linear backoff. Re-raises once attempts run out."""
55     last_error = None
56     for attempt in range(1, ATTEMPTS + 1):
57         try:
58             request = urllib.request.Request(url, headers=_headers())
59             with urllib.request.urlopen(request, timeout=TIMEOUT) as r:
60                 data = r.read()
61             return data if binary else data.decode("utf-8", "replace")
62         except urllib.error.HTTPError as e:
63             # 404 means the resource does not exist; retrying cannot help.
```

```

64         if e.code == 404:
65             raise
66         # 401/403 means the key was rejected; retrying cannot help either.
67         if e.code in (401, 403):
68             raise AuthenticationError(
69                 f"PubMLST rejected the API key (HTTP {e.code}). Check the key "
70                 f"stored with 'nextflow secrets set {API_KEY_ENV}'." from e
71         last_error = e
72     except Exception as e:
73         last_error = e
74     if attempt < ATTEMPTS:
75         delay = BACKOFF_BASE * attempt
76         print(f"[WARN] Request to {url} failed (attempt {attempt}/{ATTEMPTS}): "
77             f"{last_error}. Retrying in {delay}s...", file=sys.stderr)
78         time.sleep(delay)
79     raise RuntimeError(f"Could not fetch {url} after {ATTEMPTS} attempts: {last_error}")
80
81
82 def get_json(url):
83     return json.loads(_get(url))
84
85
86 def get_text(url):
87     return _get(url)
88
89
90 def resolve_db(species):
91     """Accept 'sagalactiae' or 'pubmlst_sagalactiae_seqdef'; return the full name."""
92     species = species.strip()
93     if species.startswith("pubmlst_") and species.endswith("_seqdef"):
94         return species
95     return f"pubmlst_{species}_seqdef"
96
97
98 def short_name(db):
99     """Inverse of resolve_db: 'pubmlst_sagalactiae_seqdef' -> 'sagalactiae'."""
100     if db.startswith("pubmlst_") and db.endswith("_seqdef"):
101         return db[len("pubmlst_"):-len("_seqdef")]
102     return db
103
104
105 def list_seqdef_databases():
106     """Return [(db_name, description)] for every seqdef database in PubMLST."""
107     groups = get_json(f"{BASE_URL}/db")
108     databases = []
109     for group in groups:
110         for db in group.get("databases", []):
111             if db["name"].endswith("_seqdef"):
112                 databases.append((db["name"], db["description"]))
113     return sorted(set(databases))
114
115
116 def detect_mlst_scheme(db, scheme_id=None):
117     """Return the full scheme record for the MLST scheme of a seqdef database.
118
119     An explicit `scheme_id` is used as given. Otherwise the scheme whose
120     description starts with 'MLST' is selected, rather than assuming id 1.
121     """
122     if scheme_id is not None:
123         scheme = get_json(f"{BASE_URL}/db/{db}/schemes/{scheme_id}")
124         scheme["_id"] = str(scheme_id)
125         return scheme
126
127     listing = get_json(f"{BASE_URL}/db/{db}/schemes")
128     for s in listing.get("schemes", []):
129         if s["description"].strip().upper().startswith("MLST"):
130             scheme = get_json(s["scheme"])
131             scheme["_id"] = s["scheme"].rstrip("/").split("/")[-1]
132             return scheme
133     raise RuntimeError(
134         f"Database '{db}' publishes no MLST scheme."

```

```

135         f"Run --list_species to see the supported species."
136     )
137
138
139     def scheme_loci(scheme):
140         """Locus names of the scheme, in the order PubMLST publishes them."""
141         return [u.rstrip("/").split("/")[-1] for u in scheme.get("loci", [])]
142
143
144     def has_profiles(scheme):
145         """True when the scheme defines STs, which is required to assign one."""
146         return bool(scheme.get("primary_key_field"))

```

5. bin/download_pubmlst.py

Descarga del esquema MLST de cada especie · 139 líneas · SHA-256

fd5f392c258c372de34bfff935749e414c80814b3542c74ce79cef91eaace9713

```
1  #!/usr/bin/env python3
2  """Download a species' MLST scheme from PubMLST and prepare the KMA inputs.
3
4  Discovering the scheme and its loci requires parsing the API's JSON, which is
5  why this runs as a script rather than a shell one-liner: neither the locus list
6  nor the scheme id can be hardcoded, as both vary between species.
7
8  For a species <sp> it writes:
9      <sp>_profiles.tsv    ST profile table
10     <sp>_loci.txt         one locus per line, in PubMLST order
11     <sp>_alleles.fasta    all alleles concatenated, ready for `kma index`
12 """
13
14 import argparse
15 import os
16 import sys
17
18 import pubmlst_api as api
19
20
21 class MissingKeyError(RuntimeError):
22     """No API key and anonymous access was not requested."""
23
24
25 # Exit code for errors that retrying cannot fix: a missing or rejected key.
26 # The pipeline maps it to 'terminate' instead of spending its retries.
27 EXIT_AUTH = 3
28
29
30 def validate_profiles(text, loci, db):
31     """Check that the profile table is usable before building the database.
32
33     Requires the scheme's loci to be present as columns. A looser check on the
34     header alone matches almost any text, including API error pages.
35     """
36     lines = [l for l in text.splitlines() if l.strip()]
37     if len(lines) < 2:
38         raise RuntimeError(f"Profile table for '{db}' is empty or has no data rows.")
39
40     columns = [c.strip() for c in lines[0].split("\t")]
41     if columns[0] != "ST":
42         raise RuntimeError(
43             f"Profile header for '{db}' does not start with 'ST': {columns[:3]}"
44         )
45
46     missing = [l for l in loci if l not in columns]
47     if missing:
48         raise RuntimeError(
49             f"Profile table for '{db}' is missing scheme loci columns: {missing}"
50         )
51     return len(lines) - 1
52
53
54 def main():
55     p = argparse.ArgumentParser(description="Download an MLST scheme from PubMLST for NAKAST.")
56     p.add_argument("--species", required=True,
57                    help="Species: 'sagalactiae' or 'pubmlst_sagalactiae_seqdef'")
58     p.add_argument("--scheme", default=None,
59                    help="Force a scheme id. Autodetected by default.")
60     p.add_argument("--outdir", default=".", help="Output directory")
61     p.add_argument("--anonymous", action="store_true",
62                    help="Query PubMLST without an API key. Only data deposited up "
```

```

63         "to 31 December 2024 is returned.")
64     args = p.parse_args()
65
66     if api.authenticated():
67         print("[INFO] PubMLST access: authenticated with a data access key.")
68     elif args.anonymous:
69         print("[WARN] PubMLST access: anonymous. Only alleles and STs deposited up "
70             "to 31 December 2024 are available; newer ones will be missing.",
71             file=sys.stderr)
72     else:
73         raise MissingKeyError(
74             f"No PubMLST API key found in {api.API_KEY_ENV}. Store it with "
75             f"'nextflow secrets set {api.API_KEY_ENV}', or pass --anonymous to use "
76             f"only data deposited up to 31 December 2024.")
77
78     db = api.resolve_db(args.species)
79     sp = api.short_name(db)
80     os.makedirs(args.outdir, exist_ok=True)
81     out_path = lambda n: os.path.join(args.outdir, n)
82
83     print(f"[INFO] PubMLST database: {db}")
84
85     scheme = api.detect_mlst_scheme(db, args.scheme)
86     scheme_id = scheme["_id"]
87     loci = api.scheme_loci(scheme)
88     if not loci:
89         raise RuntimeError(f"Scheme {scheme_id} of '{db}' declares no loci.")
90
91     print(f"[INFO] MLST scheme detected: id={scheme_id} "
92         f"({scheme.get('description','?')} with {len(loci)} loci: {' '.join(loci)})")
93
94     if not api.has_profiles(scheme):
95         raise RuntimeError(
96             f"Scheme {scheme_id} of '{db}' defines no ST profiles, "
97             f"so no Sequence Type can be assigned."
98         )
99
100     # --- ST profiles ---
101     profiles = api.get_text(
102         f"{api.BASE_URL}/db/{db}/schemes/{scheme_id}/profiles_csv")
103     n_profiles = validate_profiles(profiles, loci, db)
104     with open(out_path(f"{sp}_profiles.tsv"), "w") as fh:
105         fh.write(profiles)
106     print(f"[INFO] {n_profiles} ST profiles downloaded.")
107
108     # --- alleles, one request per locus ---
109     total = 0
110     with open(out_path(f"{sp}_alleles.fasta"), "w") as out:
111         for locus in loci:
112             fasta = api.get_text(f"{api.BASE_URL}/db/{db}/loci/{locus}/alleles_fasta")
113             if not fasta.startswith(">"):
114                 raise RuntimeError(
115                     f"Alleles for '{locus}' are not FASTA (they start with "
116                     f"'{fasta[:40]!r}').")
117             n = fasta.count(">")
118             total += n
119             print(f"[INFO] {locus}: {n} alleles")
120             out.write(fasta if fasta.endswith("\n") else fasta + "\n")
121
122     # Authoritative locus list. parse_kma.py reads it instead of guessing
123     # which columns of the profile table are loci.
124     with open(out_path(f"{sp}_loci.txt"), "w") as fh:
125         fh.write("\n".join(loci) + "\n")
126
127     print(f"[INFO] Total: {total} alleles across {len(loci)} loci for {sp}.")
128
129
130
131 if __name__ == "__main__":
132     try:
133         main()

```

```
134     except (api.AuthenticationError, MissingKeyError) as e:
135         print(f"[ERROR] {e}", file=sys.stderr)
136         sys.exit(EXIT_AUTH)
137     except Exception as e:
138         print(f"[ERROR] {e}", file=sys.stderr)
139         sys.exit(1)
```

6. bin/parse_kma.py

Llamado de alelos y asignación de ST y CC · 287 líneas · SHA-256

3222d0c7021b2a933dcf1cde2a741a7f3d72588153c2155a73a1af86890747a4

```
1  #!/usr/bin/env python3
2  import os
3  import glob
4  import pandas as pd
5  import numpy as np
6  import argparse
7
8  # Default thresholds; all are overridable from the command line.
9  MIN_DEPTH = 30.0      # Minimum depth before an allele call is trusted
10 MIN_IDENTITY = 90.0    # Minimum identity to the reference allele (%)
11 MIN_COVERAGE = 90.0   # Minimum template coverage (%)
12
13 # The number of loci is not a constant: PubMLST schemes carry between 2 and 10.
14 # The list comes from <species>_loci.txt, written by download_pubmlst.py from
15 # the scheme definition itself.
16
17 def load_loci(path):
18     with open(path) as fh:
19         loci = [l.strip() for l in fh if l.strip()]
20     if not loci:
21         raise ValueError(f"Locus file '{path}' is empty.")
22     return loci
23
24
25 def order_loci(loci, db_df):
26     """Order loci as they appear in the profile table.
27
28     That is PubMLST's canonical order, so the report columns match the official
29     scheme table.
30     """
31     if db_df is None:
32         return list(loci)
33     present = set(loci)
34     in_table = [c for c in db_df.columns if c in present]
35     return in_table + [l for l in loci if l not in in_table]
36
37
38 def split_locus_allele(template):
39     """Split '<locus>_<number>' into (locus, number).
40
41     Splits from the RIGHT: several PubMLST loci contain underscores in their
42     name ('MLST_adk_1', '16S_rRNA_1', 'int_hyp_1'). Splitting from the left
43     yields 'MLST', '16S' or 'int' as the locus and corrupts the profile
44     without raising an error.
45     """
46     for sep in ("_", "-"):
47         if sep in template:
48             locus, _, allele = template.rpartition(sep)
49             if allele.isdigit():
50                 return locus, allele
51     return template, ""
52
53
54 def process_sample(filepath, loci, depth_threshold=MIN_DEPTH, min_identity=MIN_IDENTITY, min_c
overage=MIN_COVERAGE):
55     sample_name = os.path.basename(filepath).replace(".res", "")
56
57     # KMA .res files are tab-separated with a header row.
58     try:
59         df = pd.read_csv(filepath, sep="\t")
60     except Exception as e:
61         print(f"[ERROR] Could not read {filepath}: {e}")
62     return
```

```

63
64 # KMA pads its column names with spaces
65 df.columns = df.columns.str.strip()
66
67 # Discard templates with no meaningful support
68 df_clean = df[ (df['Depth'] >= depth_threshold) ].copy()
69
70 # Split the template id into locus and allele number: 'glcK_172' becomes
71 # ('glcK', '172'). See split_locus_allele().
72 split_ids = df_clean['#Template'].apply(split_locus_allele)
73 df_clean['Locus'] = split_ids.apply(lambda t: t[0])
74 df_clean['Allele'] = split_ids.apply(lambda t: t[1])
75
76 # Pick one winning allele per locus
77 winners = annotate_alleles(df_clean, min_identity, min_coverage,
min_depth=depth_threshold)
78
79 # A profile is complete only if every locus of the scheme was recovered
80 loci_found = len(winners)
81
82 loci_expected = len(loci)
83
84 status = "OK"
85 if loci_found < loci_expected:
86     status = f"INCOMPLETE ({loci_found}/{loci_expected})"
87
88 # Order the loci as PubMLST publishes them. pd.Categorical turns any locus
89 # outside `loci` into NaN, so `loci` must belong to this species' scheme.
90 winners['Locus'] = pd.Categorical(winners["Locus"], categories=loci, ordered=True)
91
92 # Transpose into a single profile row: one column per locus
93 winners = winners.sort_values("Locus").set_index("Locus")
94 profile = winners[['Allele']].T
95 profile.insert(0, 'Sample', sample_name)
96 profile["Profile"] = status
97
98 return profile
99
100 def annotate_alleles(df, min_identity=MIN_IDENTITY, min_coverage=MIN_COVERAGE, min_depth=30, p
erfect_bonus=1.5):
101
102     df = df.copy()
103     df["Allele"] = df["Allele"].astype(str)
104
105     # Depth filter first, before any ranking
106     df = df[df["Depth"] >= min_depth].copy()
107
108     # Normalised alignment metrics
109     df["Score_Ratio"] = df["Score"] / df["Expected"]
110     df["Score_per_depth"] = df["Score"] / (df["Depth"] * df["Template_length"])
111
112     # A perfect match gets a multiplicative bonus rather than an absolute
113     # override, so it still needs depth behind it to win the locus.
114     df["_is_perfect"] = (
115         (df["Template_Identity"] == 100) &
116         (df["Template_Coverage"] == 100) &
117         (df["Query_Coverage"] == 100)
118     ).astype(int)
119
120     df["Confidence"] = (
121         df["Score_Ratio"]
122         * np.log1p(df["Depth"])
123         * (1 + df["_is_perfect"] * (perfect_bonus - 1)) # ej: 1.5x si es perfecto
124     )
125
126     # Confidence alone decides the winner
127     df = (
128         df.sort_values(
129             by=["Locus", "Confidence"],
130             ascending=[True, False]
131         )

```

```

132     .drop_duplicates(subset="Locus", keep="first")
133     .drop(columns="_is_perfect")
134     .copy()
135 )
136
137 exact_ident = df["Template_Identity"] == 100
138 exact_tcov = df["Template_Coverage"] == 100
139 qcov_exact = df["Query_Coverage"] == 100
140 qcov_ins = df["Query_Coverage"] < 100
141 qcov_trunc = df["Query_Coverage"] > 100
142 good_ident = df["Template_Identity"] >= min_identity
143 good_tcov = df["Template_Coverage"] >= min_coverage
144
145 df["Allele"] = np.select(
146     [
147         qcov_ins,
148         qcov_exact & exact_ident & exact_tcov,
149         qcov_exact & ~exact_ident & exact_tcov,
150         qcov_trunc & good_ident & good_tcov,
151         ~good_ident | ~good_tcov,
152     ],
153     [
154         "INS",
155         df["Allele"],
156         "~" + df["Allele"],
157         df["Allele"] + "?",
158         "-",
159     ],
160     default="-"
161 )
162
163 return df
164
165 def assign_st(df, db_df, genes):
166     """Assign ST and CC by matching the allelic signature against PubMLST.
167
168     `genes` is the scheme's locus list and `db_df` the profile table already
169     loaded with dtype=str (see read_profiles). The match is exact: an annotated
170     allele such as '~9' matches no profile, which is intended.
171     """
172     if db_df is None:
173         print("[INFO] No MLST profile table supplied; skipping ST assignment.")
174         df.insert(1, 'ST', '-')
175         df.insert(2, 'CC', '-')
176         return df
177
178     missing_in_table = [g for g in genes if g not in db_df.columns]
179     if missing_in_table:
180         print(f"[ERROR] The profile table has no columns for loci {missing_in_table}. "
181               f"Skipping ST assignment.")
182         df.insert(1, 'ST', '-')
183         df.insert(2, 'CC', '-')
184         return df
185
186     db_df = db_df.copy()
187     db_df["signature"] = db_df[genes].fillna("-").astype(str).agg("/".join, axis=1)
188
189     missing_genes = [g for g in genes if g not in df.columns]
190     if missing_genes:
191         df = df.copy()
192         df[missing_genes] = "-"
193     df["signature"] = df[genes].astype(str).agg("/".join, axis=1)
194
195     # Detect the clonal complex column across naming variants
196     cc_aliases = {"clonal_complex", "clonal complex", "cc", "clonalcomplex"}
197     cc_col = next((c for c in db_df.columns if c.strip().lower() in cc_aliases), None)
198     cols = ["ST", "signature"] + ([cc_col] if cc_col else [])
199
200     out = df.merge(db_df[cols], on="signature", how="left")
201

```

```

202 out["ST"] = out["ST"].apply(lambda x: str(int(x)) if pd.notna(x) and x != "-" else "-") #
drop decimals from ST and fill unmatched rows with '-'
203
204 if cc_col:
205     out = out.rename(columns={cc_col: "CC"})
206     out["CC"] = out["CC"].fillna("-")
207 else:
208     out["CC"] = "-"
209
210 order = ["Sample", "ST", "CC"] + list(genes) + ["Profile"]
211
212 order = [c for c in order if c in out.columns]
213 return out[order].replace("", pd.NA).fillna("-")
214
215 def read_profiles(path):
216     """Read the PubMLST profile table.
217
218     dtype=str matters: with a blank cell anywhere in a locus column pandas
219     would infer floats, the signature would read '1.0/1/2/...', and every
220     profile match would fail silently, returning ST='-' for all samples.
221     """
222     if path is None:
223         return None
224     try:
225         return pd.read_csv(path, sep="\t", dtype=str)
226     except Exception as e:
227         print(f"[ERROR] Could not read the MLST profile table: {e}")
228         return None
229
230
231 def main():
232     parser = argparse.ArgumentParser(description="Call MLST alleles from KMA results.")
233     parser.add_argument("-i", "--input_folder", help="Directory containing .res files", require
d=True)
234     parser.add_argument("-p", "--profiles", required=False, help="PubMLST profile table (TSV)")
235     parser.add_argument("-l", "--loci", required=False,
236                         help="File with one locus per line (<species>_loci.txt). "
237                             "If omitted, loci are inferred from the profile table.")
238     parser.add_argument("-o", "--output", default="mlst_profiles",
239                         help="Base name for the .txt and .xlsx reports")
240     parser.add_argument("-d", "--min_depth", type=int, default=MIN_DEPTH, help="Minimum depth
to consider an allele")
241     parser.add_argument("--min_identity", type=float, default=MIN_IDENTITY, help="Minimum
identity to consider an allele")
242     parser.add_argument("-c", "--min_coverage", type=float, default=MIN_COVERAGE,
243                         help="Minimum coverage to consider an allele")
244     args = parser.parse_args()
245
246     db_df = read_profiles(args.profiles)
247
248     # Locus list. Preferred source is the file written by download_pubmlst.py.
249     # Failing that, infer it from the profile table by dropping ST and the
250     # metadata columns PubMLST may append (clonal_complex, species, ...).
251     if args.loci:
252         loci = load_loci(args.loci)
253     elif db_df is not None:
254         non_locus = {"st", "clonal_complex", "species", "mlst_clade", "lineage", ""}
255         loci = [c for c in db_df.columns if c.strip().lower() not in non_locus]
256     else:
257         parser.error("Either --loci or --profiles is required to know the scheme loci.")
258
259     loci = order_loci(loci, db_df)
260     print(f"[INFO] Scheme with {len(loci)} loci: {' '.join(loci)}")
261
262     # One .res file per sample
263     files = sorted(glob.glob(os.path.join(args.input_folder, "*.res")))
264
265     profiles = []
266     for f in files:

```

```

266     profile = process_sample(f, loci, depth_threshold=args.min_depth, min_identity=args.mi
n_identity, min_coverage=args.min_coverage)
267     if profile is not None and not profile.empty:
268         profiles.append(profile)
269
270     if not profiles and args.input_folder:
271         # Both formats are written even when empty: GENERATE_REPORT declares
272         # .txt and .xlsx as outputs and fails if either is missing.
273         empty = pd.DataFrame(columns=["Sample", "ST", "CC"] + list(loci) + ["Profile"])
274         empty.to_csv(f"{args.output}.txt", sep="\t", index=False)
275         empty.to_excel(f"{args.output}.xlsx", index=False)
276         print(f"[INFO] No MLST profiles generated. Empty reports written to {args.output}.")
277         return
278
279     profiles_df = pd.concat(profiles, ignore_index=True)
280     st_cc_assign_df = assign_st(profiles_df, db_df, loci)
281
282     st_cc_assign_df.sort_values(by='Sample').to_csv(f"{args.output}.txt", sep="\t",
index=False)
283     st_cc_assign_df.sort_values(by='Sample').to_excel(f"{args.output}.xlsx", index=False)
284     print(f"[INFO] MLST profiles written to {args.output}")
285
286 if __name__ == "__main__":
287     main()

```

7. bin/list_pubmlst_species.py

Catálogo de especies compatibles · 147 líneas · SHA-256

737db11a6337daf224d2d8341e1884716e63ee7a1807f9aa411295a51c99f2e7

```
1  #!/usr/bin/env python3
2  """Catalogue the PubMLST species that NAKAST supports.
3
4  Supported means the seqdef database publishes an MLST scheme with defined ST
5  profiles (a primary key), which is what the pipeline needs to assign a
6  Sequence Type.
7
8  Two modes:
9      --tabla                query PubMLST live and print the catalogue
10     --write-catalogue D    regenerate assets/pubmlst_species.tsv and
11                             docs/supported_species.md under directory D
12 """
13
14 import argparse
15 import concurrent.futures as cf
16 import os
17 import sys
18
19 import pubmlst_api as api
20
21
22 def inspect_database(item):
23     """Return the catalogue record for a database, or None if it has no MLST."""
24     db, desc = item
25     try:
26         scheme = api.detect_mlst_scheme(db)
27     except Exception:
28         return None
29     if not api.has_profiles(scheme):
30         return None
31     loci = api.scheme_loci(scheme)
32     if not loci:
33         return None
34     # Clonal complex is optional in PubMLST; NAKAST reports '-' when a scheme
35     # does not define it. `fields` is a list of URLs whose last path segment is
36     # the field name.
37     fields = {str(u).rstrip("/").split("/")[-1].strip().lower()
38               for u in scheme.get("fields", [])}
39     return {
40         "species": api.short_name(db),
41         "description": desc.replace(" sequence/profile definitions", "").strip(),
42         "db": db,
43         "scheme_id": scheme["_id"],
44         "n_loci": len(loci),
45         "loci": " ".join(loci),
46         "clonal_complex": "yes" if "clonal_complex" in fields else "no",
47     }
48
49
50 def build_catalogue(workers=6):
51     databases = api.list_seqdef_databases()
52     print(f"[INFO] Querying {len(databases)} PubMLST seqdef databases...", file=sys.stderr)
53     rows = []
54     with cf.ThreadPoolExecutor(max_workers=workers) as ex:
55         for r in ex.map(inspect_database, databases):
56             if r:
57                 rows.append(r)
58     rows.sort(key=lambda r: r["description"].lower())
59     print(f"[INFO] {len(rows)} supported species out of {len(databases)} databases.",
60           file=sys.stderr)
61     return rows
62
```

```

63 COLUMNS = ["species", "description", "db", "scheme_id", "n_loci", "clonal_complex", "loci"]
64
65
66 def write_tsv(rows, path):
67     with open(path, "w") as fh:
68         fh.write("\t".join(COLUMNS) + "\n")
69         for r in rows:
70             fh.write("\t".join(str(r[c]) for c in COLUMNS) + "\n")
71
72
73 def write_md(rows, path):
74     with_cc = sum(1 for r in rows if r["clonal_complex"] == "yes")
75     not_seven = sum(1 for r in rows if r["n_loci"] != 7)
76     other_scheme = sum(1 for r in rows if r["scheme_id"] != "1")
77     with open(path, "w") as fh:
78         fh.write("# Species supported by NAKAST\n\n")
79         fh.write(
80             "Generated automatically by `bin/list_pubmlst_species.py` from the\n"
81             "PubMLST REST API (<https://rest.pubmlst.org>).\n\n"
82             "A species is supported when its `seqdef` database publishes an MLST\n"
83             "scheme with defined ST profiles, which is what NAKAST needs to assign\n"
84             "a Sequence Type.\n\n"
85         )
86         fh.write("## Summary\n\n")
87         fh.write(f"- **{len(rows)} supported species**\n")
88         fh.write(f"- {with_cc} define a Clonal Complex; the rest report `CC = -`\n")
89         fh.write(f"- {not_seven} use a number of loci other than 7\n")
90         fh.write(f"- {other_scheme} use a `scheme_id` other than 1 (NAKAST detects it)\n\n")
91         fh.write("## Usage\n\n")
92         fh.write("The value in the `species` column below is what goes in the\n")
93         fh.write("`species` column of the samplesheet:\n\n")
94         fh.write("```\nsample_id\tfastq_dir\tspecies\n")
95         fh.write("M1\tdata/M1\tSagalactiae\n```\n\n")
96         fh.write("To query the list live:\n\n")
97         fh.write("```\nbash\n\nnextflow run nakast.nf --list_species\n```\n\n")
98         fh.write("## Catalogue\n\n")
99         fh.write("| Organism | `species` | PubMLST database | Scheme | Loci | CC | \n")
100        fh.write("| ---|---|---|---|---|:--:|\n")
101        for r in rows:
102            fh.write(f"| {r['description']} | {r['species']} | {r['db']} | "
103                    f"{r['scheme_id']} | {r['n_loci']} | {r['clonal_complex']} | \n")
104        fh.write("\n## Loci per scheme\n\n")
105        for r in rows:
106            fh.write(f"- **{r['species']}** ({r['n_loci']}: {r['loci'].replace(',', ' ')}\n\n")
107
108
109 def main():
110     p = argparse.ArgumentParser(description="PubMLST species supported by NAKAST.")
111     p.add_argument("--tabla", action="store_true",
112                   help="Print the live catalogue to stdout")
113     p.add_argument("--write-catalogue", metavar="DIR", default=None,
114                   help="Regenerate assets/ and docs/ under the given repository directory")
115     args = p.parse_args()
116
117     rows = build_catalogue()
118
119     if args.write_catalogue:
120         root = args.write_catalogue
121         os.makedirs(os.path.join(root, "assets"), exist_ok=True)
122         os.makedirs(os.path.join(root, "docs"), exist_ok=True)
123         tsv = os.path.join(root, "assets", "pubmlst_species.tsv")
124         md = os.path.join(root, "docs", "supported_species.md")
125         write_tsv(rows, tsv)
126         write_md(rows, md)
127         print(f"[INFO] Wrote {tsv}")
128         print(f"[INFO] Wrote {md}")
129         return
130
131     # Table mode, used by --list_species
132     print(f"{'species':<28} {'scheme':>7} {'loci':>5} {'CC':<3} organism")

```

```

133     print("-" * 100)
134     for r in rows:
135         print(f"{r['species']:<28} {r['scheme_id']:>7} {r['n_loci']:>5} "
136               f"{r['clonal_complex']:<3} {r['description']}")
137     print("-" * 100)
138     print(f"{len(rows)} supported species. "
139           f"Use the value in the 'species' column in your samplesheet.")
140
141
142 if __name__ == "__main__":
143     try:
144         main()
145     except Exception as e:
146         print(f"[ERROR] {e}", file=sys.stderr)
147         sys.exit(1)

```